UTM UNIVERSITI TEKNOLOGI MALAYSIA | Faculty of Computing

Semester 01 2025/2026

Subject : Database Programming (SECP3623)

Section : Section 1

Task : Project II

Name	Matric No.
JOANNE CHING YIN XUAN	A23CS0227
CHUA JIA LIN	A23CS0069
EVELYN GOH YUAN QI	A23CS0222
LIM YU HAN	A23CS0241

Youtube Link: https://youtu.be/4Kv0toRyghQ

Table of Contents

1.0 Introduction

Project Title:

Event Management and Ticket Booking System

Project Description

This project implements a NoSQL environment for the Event Management and Ticket Booking System using MongoDB. The system is used to manage users, events, ticket bookings, and reviews from users after attending an event. The system supports operations such as registering new users, creating new events, booking tickets, and submitting feedback.

Unlike traditional relational databases like SQL, the system uses collections, documents, arrays, and references to store data flexibly. This allows the system to handle different data structures and high volumes of data, making it suitable for real-life event management and ticket booking scenarios.

Project Objectives

1. Create a NoSQL data model to manage users, events, bookings, and reviews.
2. Implement CRUD operations (Create, Read, Update, Delete) for all collections.
3. Demonstrate advanced MongoDB operations, including indexing, aggregation pipelines, and sorting.
4. Create meaningful queries for analytics.
5. Analyze NoSQL security considerations and design challenges.

Team Members & Roles

Team Member	Role
Chua Jia Lin	- Design users collections schema - Perform CRUD operations in collections - Perform aggregation operation in collections
Joanne Ching Yin Xuan	- Design reviews collections schema - Perform CRUD operations in collections - Perform aggregation and sorting operations in collections
Evelyn Goh Yuan Qi	- Design bookings collections schema - Perform CRU operations in collections - Perform indexing and aggregation operations in collections
Lim Yu Han	- Design events collections schema - Perform CRU operations in collections - Perform indexing and sorting operations in collections

2.0 NoSQL Data Model

List and explanation of collection

The system of Event Management and Ticket Booking is developed based on NoSQL data model of MongoDB. The system is designed into four primary collections, which are users, events, bookings and reviews. All collections are important modules of the system and are designed to facilitate a flexible data storage, efficient querying, and operations of real-world event management.

The users collection is used to store a user profile such as name, email, interests, address and date account was created. This set helps to register users and monitor their personal preferences. The events collection stores data pertaining to the events like event title, category, price, date, capacity and location. The bookings collection is where the ticket purchase information is stored by connecting users to events, the quantity of tickets purchased, price, and booking status. The reviews collection is a collection of feedback given out by the users who attended events in the form of ratings and comments.

Example JSON documents

User Document


```
{
  "_id": "u1",
  "name": "Ali Hassan",
  "email": "ali@gmail.com",
  "interests": ["music", "tech"],
  "address": {
    "city": "KL",
    "country": "MY"
  },
  "createdAt": "2025-12-01"
}
```


This document represents a user profile. The interests field is stored as an array, while the address field is an embedded document containing city and country information.

Booking Document


```
{
  "userId": "u1",
  "eventId": "e1",
  "tickets": 2,
  "totalPrice": 300,
  "status": "confirmed"
}
```


This document represents a booking record. The userId and eventId fields reference documents from the users and events collections, establishing relationships between users and events.

Explanation of embedding vs referencing

Embedding is done where the data is closely tied and is often accessed together like when the address information is embedded in the user document and location information in the event document. This will enhance the speed of reading, as fewer queries are required. Then, referencing is used when the data has to be spread over various collections, including connecting user IDs and event IDs in the bookings and reviews collections. This assists in minimizing duplication of data and the collection of logical relationships between collections.

Data types used

The system has different data types, which effectively reflect real-world entities. Name, category and comments are done using strings. Ticket prices, capacities, ratings and quantities of tickets are done in terms of numbers. Event schedules and account creation time is in form of dates. Storing of multiple values like user interests is done by arrays whereas embedded documents like addresses and locations are kept in objects. Such a mixture of data types contributes to the flexibility and scalability of the NoSQL database design.

3.0 CRUD Implementation Summary

3.1 Insert Operations

insertOne()

Figure 3.1 shows the command of inserting a new user into the "users" collection using the insertOne() command, while Figure 3.2 shows the output of the insertOne() command.

```
> db.users.insertOne({
    _id:"u1",
    name:"Ali Hassan",
    email:"ali@gmail.com",
    interests:["music","tech"],
    address:{city:"KL",country:"MY"},
    createdAt:new Date("2025-12-01")
  })
```

Figure 3.1 Command of inserting a new user using insertOne()

```
< {
    acknowledged: true,
    insertedId: 'u1'
  }
```

Figure 3.2 Output of inserting a new user using insertOne()

insertMany()

Figure 3.3 shows the command of inserting multiple reviews into the “reviews” collection using the insertMany() command, while Figure 3.4 shows the output of the insertMany() command.

```
> db.reviews.insertMany([
    {
      userId:"u4",
      eventId:"e1",
      rating:4,
      comment:"Very fun"
    },
    {
      userId:"u5",
      eventId:"e2",
      rating:4,
      comment:"Useful"
    },
    {
      userId:"u6",
      eventId:"e4",
      rating:5,
      comment:"Loved it"
    },
    {
      userId:"u8",
      eventId:"e6",
      rating:3,
      comment:"Good"
    },
    {
      userId:"u9",
      eventId:"e7",
      rating:1,
      comment:"It's a mess"
    },
    {
      userId:"u10",
      eventId:"e8",
      rating:5,
      comment:"Amazing"
    },
    {
      userId:"u3",
      eventId:"e5",
      rating:4,
      comment:"Good"
    },
    {
      userId:"u5",
      eventId:"e7",
      rating:4,
      comment:"Enjoyed"
    },
    {
      userId:"u7",
      eventId:"e9",
      rating:2,
      comment:"Boring"
    }
])
```

Figure 3.3 Command of inserting multiple reviews using insertMany()

```
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('6961ba30126d85249da7b9a8'),
      '1': ObjectId('6961ba30126d85249da7b9a9'),
      '2': ObjectId('6961ba30126d85249da7b9aa'),
      '3': ObjectId('6961ba30126d85249da7b9ab'),
      '4': ObjectId('6961ba30126d85249da7b9ac'),
      '5': ObjectId('6961ba30126d85249da7b9ad'),
      '6': ObjectId('6961ba30126d85249da7b9ae'),
      '7': ObjectId('6961ba30126d85249da7b9af'),
      '8': ObjectId('6961ba30126d85249da7b9b0')
    }
  }
```

Figure 3.4 Output of inserting multiple reviews using insertMany()

3.2 Query Operations

$gt

Figure 3.5 shows the command of find () query that filters review records with ratings greater than 4 using the $gt operator, while Figure 3.6 displays the resulting review documents that meet the specified criteria.

```
> db.reviews.find({rating: { $gt: 4 }})
```

Figure 3.5 Command of filters review records using $gt()

Figure 3.6 Output of filters review records using $gt()

$in

Figure 3.7 shows the use of the $in operator to retrieve event records whose category belongs to Technology, Music, and Art. This allows multiple categories to be filtered in a single query. Figure 3.8 displays the events that match the selected categories.

```
> db.events.find({
    category: { $in: ["Technology", "Music", "Art"] }
  })
```

Figure 3.7 Command of retrieving event records using $in()

Figure 3.8 Output of retrieving event records using $in()

Array Queries

Figure 3.7 shows the find() command used to retrieve users whose interests array contains the value "music". This query searches within the array field and returns all user documents that have "music" as one of their interests. Figure 3.8 displays the resulting users documents that have "music" listed in their interests.

```
> db.users.find({ interests: "music"})
```

Figure 3.9 Command for filtering users based on the interests array containing "music".

Figure 3.10 Output of users whose interests array contains "music"

Projection (Include)

Figure 3.11 shows the use of projection to include only selected fields from the users collection. This query returns only the name and email fields while excluding unnecessary data. Figure 3.12 shows the output with the specified fields displayed.

```
> db.users.find({},{ name: 1, email: 1, _id: 0 })
```

Figure 3.11 Command of retrieving name and email fields only

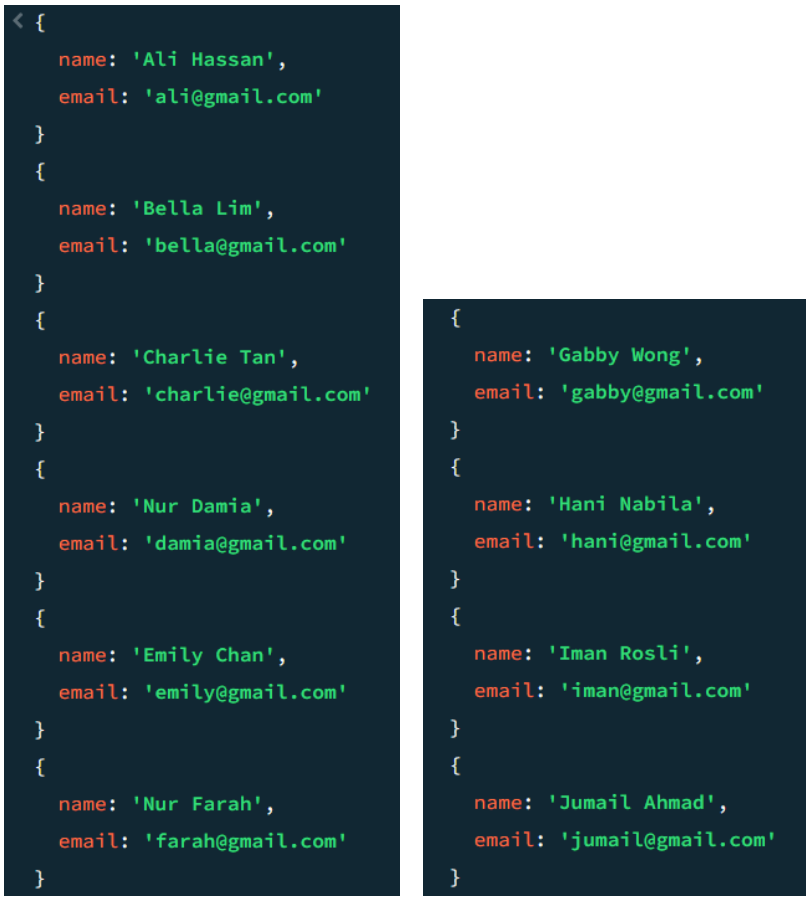

Figure 3.12 Output with the name and email fields displayed by using include

3.3 Update Operations

$set

Figure 3.13 shows the command of updating the status field of a specific booking record to "confirmed" based on the specified user ID and event ID. Figure 3.14 also shows the output of the updateOne() command, indicating that one matching document was successfully found and modified.

```
> db.bookings.updateOne(
    { userId: "u3", eventId: "e3" },
    { $set: { status: "confirmed" } }
  )
```

Figure 3.13 Command of updating the status field of a booking record using updateOne()

```
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

Figure 3.14 Output of updating the status field of a booking record using updateOne()

$inc

Figure 3.15 shows the command of adding the number of tickets and the total price paid into the "bookings" collection using the updateOne() command, while Figure 3.16 shows the output of the updateOne() command.

```
> db.bookings.updateOne(
    {userId: "u2",eventId:"e2"},
    {$inc:{tickets:1,totalPrice:120}}
  )
```

Figure 3.15 Command of updating a booking document using updateOne()

```
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

Figure 3.16 Output of updating a booking document using updateOne()

$push

Figure 3.17 shows the updateOne() command that uses the $push operator to add a new value ("gaming") into the interests array of user u4 in the users collection. This demonstrates how MongoDB can update array fields dynamically without overwriting existing data. Figure 3.18 shows the output of the updateOne() command, which shows that "gaming" has been added to the interests array for user u4.

```
> db.users.updateOne(
    { _id: "u4" },
    { $push: { interests: "gaming" } }
  )
```

Figure 3.17 Command for adding a new interest to a user using the $push operator.

```
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
> db.users.find({ _id: "u4"})
< {
    _id: 'u4',
    name: 'Nur Damia',
    email: 'damia@gmail.com',
    interests: [
      'tech',
      'gaming'
    ],
    address: {
      city: 'KL',
      country: 'MY'
    },
    createdAt: 2025-12-04T00:00:00.000Z
  }
```

Figure 3.18 Output of updated interests array after applying the $push operation.

3.4 Delete Operations

$deleteOne()

Figure 3.x shows the command of deleting a single booking record where the user ID is “u2”, the event ID is “e10”, and the booking status is “cancelled”, while Figure 3.x shows the output of the deleteOne() command and findOne() query is used to make sure the booking record no longer exists in the database.

```
> db.bookings.deleteOne({
    userId: "u2",
    eventId: "e10",
    status: "cancelled"
  })
```

Figure 3.19 Command of deleting document using deleteOne()

```
< {
    acknowledged: true,
    deletedCount: 1
  }
> db.bookings.findOne({
    userId: "u2",
    eventId: "e10",
    status: "cancelled"
  })
< null
```

Figure 3.20 Output of deleting document using deleteOne() and findOne() for checking

$deleteMany()

Figure 3.x shows the command of deleting all documents with status “cancelled” from the “bookings” collection using the deleteMany() command, while Figure 3.x shows the output of the deleteMany() command.

```
> db.bookings.deleteMany({status:"cancelled"})
```

Figure 3.21 Command of deleting multiple documents using deleteMany()

```
< {
    acknowledged: true,
    deletedCount: 6
  }
> db.bookings.find({status:"cancelled"})
<
```

Figure 3.22 Output of deleting multiple documents using deleteMany()

4.0 Advanced Operations

4.1 Indexing Operations

Single-field index

Figure 4.1 shows the command of creating a single-field index on the status field in the bookings collection. This index is designed to improve the performance of queries that filter bookings based on their status, such as retrieving confirmed or cancelled bookings. Figure 4.2 displays the output of the indexes on the bookings collection.

```
> db.bookings.createIndex({ status: 1 })
```

Figure 4.1 Command for creating a single-field index using createIndex()

```
< status_1
> db.bookings.getIndexes()
< [
    { v: 2, key: { _id: 1 }, name: '_id_' },
    { v: 2, key: { status: 1 }, name: 'status_1' }
  ]
```

Figure 4.2 Output showing the indexes on the bookings collection

Compound Index

Figure 4.3 shows the creation of a compound index on category and date fields in the events collection. This index improves query performance when filtering or sorting events based on more than one attribute. Figure 4.4 displays the list of indexes created for the collection.

```
> db.events.createIndex({ category: 1, date: 1 })
```

Figure 4.3 Command of creating compound index using createIndex()

```
< category_1_date_1
> db.events.getIndexes()
< [
    { v: 2, key: { _id: 1 }, name: '_id_' },
    { v: 2, key: { category: 1, date: 1 }, name: 'category_1_date_1' }
  ]
```

Figure 4.4 Output showing the indexes on the events collection by using getIndex()

4.2 Aggregation Operations

$match

Figure 4.5 shows the aggregation pipeline used to calculate the total number of confirmed bookings. The pipeline first applies the $match stage to filter booking records with the status set to "confirmed", and then uses the $group stage to count the total number of confirmed bookings. Figure 4.6 displays the output, which shows the total number of confirmed bookings in the system.

```
> db.bookings.aggregate([
    { $match: { status: "confirmed" } },
    { $group: { _id: null, totalConfirmedBookings: { $sum: 1 } } }
  ])
```

Figure 4.5 Command using $match and $group to calculate total number of confirmed bookings

```
< {
    _id: null,
    totalConfirmedBookings: 13
  }
```

Figure 4.6 Output that shows the total number of confirmed bookings in the system.

$limit

Figure 4.7 shows the aggregation pipeline used to retrieve the top three most expensive events from the events collection. The pipeline first sorts the events by price in descending order and then $limit operation is applied to return only the top three records. Figure 4.8 displays the resulting event documents, which include the top 3 most expensive events

```
> db.events.aggregate([
    { $sort: { price: -1 } },
    { $limit: 3 }
  ])
```

Figure 4.7 Command using $sort and $limit to retrieve top three most expensive events

Figure 4.8 Output that shows the top three most expensive events based on price

4.3 Sorting Operations

Ascending Order

Figure 4.7 shows the sorting of event records in ascending order based on price field. This allows events to be arranged from the lowest value to the highest value. Figure 4.8 displays the sorted output in ascending order.

```
> db.events.find().sort({ price: 1 })
```

Figure 4.9 Command of sorting event price by using sort()

Figure 4.10 Output of sorted event based on price

4.4 Query Performance Discussion – How Indexes Improve Speed

Indexes significantly improved the query performance by allowing MongoDB to locate data without scanning every document in the collection. Without an index, MongoDB will perform a collection scan by checking each document one by one, which reduces the query speed as data increases.

When an index is created:

- MongoDB builds a sorted data structure based on the field indexed.
- Queries can jump directly to the matching records.
- This reduces disk reads and CPU usage.

To sum up, indexes make the read operations faster. However, it slightly increases the storage usage and slows down the insert and update operations because the index must also be updated when changes happen.

4.5 Challenges Encountered

During the development of the Event Management and Ticket Booking System using MongoDB, there are several challenges that we have encountered. One of the challenges is to make sure the correct use of MongoDB commands, as even minor syntax errors in insert, update or query operations could lead to data inaccuracies or loss. Next, we also faced challenges in handling arrays, nested documents and references across different collections, which we need to be aware of the use of operators and dot notation to accurately retrieve and update data. Furthermore, as MongoDB does not enforce foreign key constraints, it was challenging when maintaining data consistency across users, events, bookings, and reviews. In addition, when working on a shared database, update and deletion operations require extra caution to prevent accidental data loss.

5.0 Security & Limitations

Basic Access Controls

MongoDB supports Role-Based Access Control (RBAC), enabling administrators to define distinct roles such as read-only, read-write, or administrator users. When deploying an Event Management and Ticket Booking System in real practice, access restrictions can be implemented to ensure only authorized users or applications can insert, update, or delete booking and user data. This can help to prevent unauthorized modifications to sensitive data like booking information, payment records, and user profiles.

Injection Risk

Although MongoDB does not use SQL, it remains vulnerable to NoSQL injection attacks if user input is not properly validated. For instance, malicious query filter inputs can be used to bypass authentication or data tampering. To mitigate this risk, applications should always validate and sanitize user input and avoid using untrusted input directly in database queries.

Database Constraints or Limitations

By default, MongoDB does not enforce strict schema rules, meaning that without careful management, the system may store inconsistent or incomplete data. For instance, missing fields or incorrect data types could lead to discrepancies in booking or event records. Furthermore, MongoDB does not provide built-in foreign key constraints, so applications must manually maintain referential relationships between collections such as userID and eventID to ensure consistency of data.

6.0 Teamwork

In this project, the team members collaborated effectively to design and implement a NoSQL environment for an Event Management and Ticket Booking System using MongoDB. Each member contributed to the CRUD operations, indexing, aggregation pipelines, and report preparation.

Member Contributions

Team Member	Responsibilities & Contributions
Chua Jia Lin	- Designed user collection schemas - Implemented CRUD and aggregation operations in collections - Analyzed the effect of indexing on query performance - Prepared the Introduction and Teamwork section in report - Explained and pasted the output of the commands for CRUD and aggregation operations in the report.
Joanne Ching Yin Xuan	- Designed reviews collection schemas - Performed Create, Read, Update, and Delete (CRUD) operations in the collections with the application of conditional query filters to enable precise data retrieval. - Implemented aggregation pipelines using multiple stages to generate summarized analytical results. - Demonstrated sorting operations in descending order to organize query outputs effectively. - Explained and included screenshots of RUD, aggregation commands, and their outputs in the report. - Conclude what was learned, challenges, and recommendations for improvements.
Evelyn Goh Yuan Qi	- Designed bookings collection schemas - Performed Create ,Read and Update (CRU) operations in the collections.

	- Implemented indexing and aggregation operations to support data analysis and improve query performance. - Analyzed and documented the challenges encountered during database development. - Explained the security considerations and system limitations of the database design. - Explained and included screenshots of RU, indexing and aggregation commands and their outputs in the report.
Lim Yu Han	- Designed events collection schemas - Performed Create, Read and Update (CRU) operations such as $in and projection. - Implemented bulk update operations using updateMany() to modify multiple documents efficiently. - Created a compound index to improve query performance for multi-field filtering. - Demonstrated sorting operations in ascending order to organize event records logically. - Contributed to the NoSQL data model explanation and documented query logic in the report.

Collaboration Strategy

At the beginning of the collaboration, every task was divided equally and fairly to all members. This ensured that every member gained hands-on experience with the CRUD, aggregation, indexing, and sorting operations.

At the development stage, the team used MongoDB Atlas as a shared cloud database, allowing all members to work on their parts simultaneously. Besides that, a .js file that stores all members' works was created as a backup to prevent accidental data loss, especially during the delete operations.

7.0 Conclusion

This project gave us hands-on work in building a NoSQL backend for an Event Management and Ticket Booking System using MongoDB. We learned how to use the document data model by making collections with embedded documents, arrays, references, and the right data types. We also learned how to do basic CRUD actions. These include finding data with find() and findOne() using conditions, updating records with update commands, and deleting documents safely. We used aggregation pipelines, sorting, and indexing to see how MongoDB helps with data analysis and speeds up queries.

Several challenges were encountered during the development process. Coordinating updates and deleting operations in a shared database environment requires extra caution to prevent unintended data loss. Furthermore, inaccurate or unexpected query outputs highlighted the importance of carefully designing query conditions, aggregation stages, and maintaining data consistency across collections. Resolving these issues involved verifying intermediate query results, refining filter conditions, and validating stored data.

To address these challenges, future improvements could include implementing schema validation to reduce inconsistent data entries and improve data integrity. More structured testing procedures can also be applied before executing to update or delete operations to minimize errors. Additionally, optimizing indexing strategies based on actual query patterns would further enhance performance and reliability. Overall, this project reinforced our understanding of NoSQL database concepts while strengthening practical skills in MongoDB backend development for real-world non-relational systems.